

Javascript Events

- Events are actions or occurrences that happen in the system when you are programming, which the system tells you about so you can respond to them in some way if desired.
- For example, if the user selects a button on a webpage, you might want to respond to that action by displaying an information box.

Examples

- The user selects a certain element or hovers the cursor over a certain element.
- The user chooses a key on the keyboard.
- The user resizes or closes the browser window.
- A web page finishes loading.
- A form is submitted.
- A video is played, paused, or finishes.
- An error occurs.

Event	Value	Description
onchange	script	Script runs when the element changes
onsubmit	script	Script runs when the form is submitted
onreset	script	Script runs when the form is reset
onselect	script	Script runs when the element is selected
onblur	script	Script runs when the element loses focus
onfocus	script	Script runs when the element gets focus
onkeydown	script	Script runs when key is pressed
onkeypress	script	Script runs when key is pressed and released
onkeyup	script	Script runs when key is released
onclick	script	Script runs when a mouse click
ondblclick	script	Script runs when a mouse double-click
onmousedown	script	Script runs when mouse button is pressed
onmousemove	script	Script runs when mouse pointer moves
onmouseout	script	Script runs when mouse pointer moves out of an element
onmouseover	script	Script runs when mouse pointer moves over an element
onmouseup	script	Script runs when mouse button is released

Event Handling

- **Event is an activity** that represents a change in the environment
- When JavaScript is used in HTML pages, JavaScript can "react" on these events. Example - mouse clicks, particular key of keyboard

HTML Events

An HTML event can be something the browser does, or something a user does.

Here are some examples of HTML events:

- An HTML web page has finished loading
- An HTML input field was changed
- An HTML button was clicked

Event Handling

- allow scripts to respond to user interactions and modify the page accordingly
- Events and event handling help to make web applications more dynamic and interactive.

Event Handler

An event handler is a function that responds to an event.

Registering an Event Handler

Assigning an event handler to an event for a DOM node

Registering an Event - addEventListener

- addEventListener () is available for every DOM node.
- The method takes three arguments:
 - The first is the **name of the event** for which we're registering a
 - handler.
 - The second is the **function** that will be called to handle the event.
 - The last argument is **usecapture**. (true→capturing phase, false→bubbling phase)

Capturing phase - the event goes down to the element.

Bubbling phase - the event bubbles up from the element

Syntax:

```
addEventListener("Event", Function, usecapture);
```

```
removeEventListener("event", function, usecapture);
```

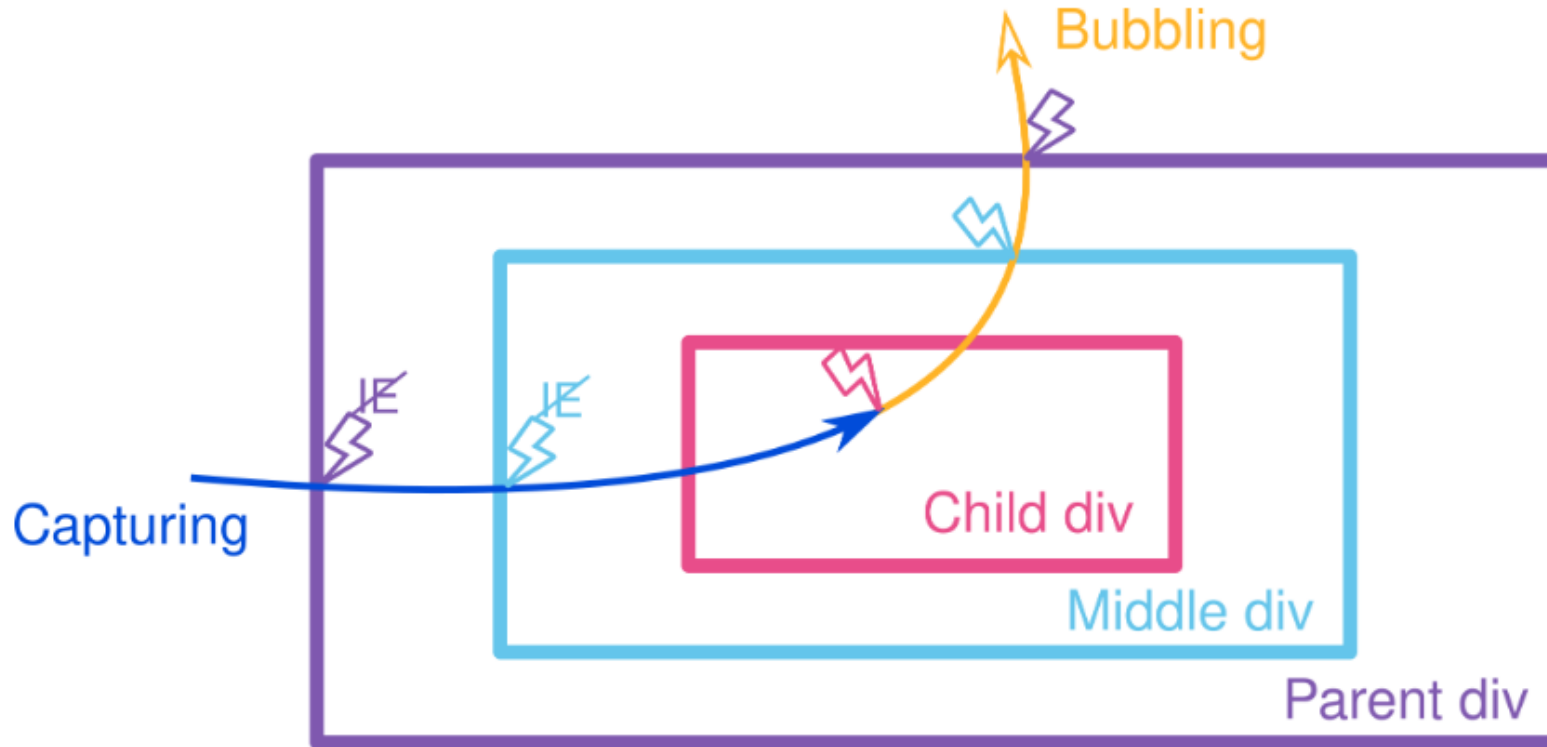
Example:

```
window.addEventListener( "load", startTimer, false );
```

```
window.removeEventListener( "load", startTimer, false );
```

Event capturing- The event in outer element should be handled first and then event in the inner element should get handled

Event bubbling- The event in inner element should be handled first and then event in the outer element should get handled



Common HTML Events

Here is a list of some common HTML events:

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

Event	Description
abort	Fires when image transfer has been interrupted by user.
change	Fires when a new choice is made in a select element, or when a text input is changed and the element loses focus.
click	Fires when the user clicks the mouse.
dblclick	Fires when the user double clicks the mouse.
focus	Fires when a form element gets the focus.
keydown	Fires when the user pushes down a key.
keypress	Fires when the user presses then releases a key.
keyup	Fires when the user releases a key.
load	Fires when an element and all its children have loaded.

<code>mousedown</code>	Fires when a mouse button is pressed.
<code>mousemove</code>	Fires when the mouse moves.
<code>mouseout</code>	Fires when the mouse leaves an element.
<code>mouseover</code>	Fires when the mouse enters an element.
<code>mouseup</code>	Fires when a mouse button is released.
<code>reset</code>	Fires when a form resets (i.e., the user clicks a reset button).
<code>resize</code>	Fires when the size of an object changes (i.e., the user resizes a window or frame).
<code>select</code>	Fires when a text selection begins (applies to <code>input</code> or <code>textarea</code>).
<code>submit</code>	Fires when a form is submitted.
<code>unload</code>	Fires when a page is about to unload.

Onload

```
<!DOCTYPE html>
```

```
<html>
```

welcome

```
<head>
```

```
<script>
```

```
function start()
```

```
{
```

```
    window.alert("welcome");
```

```
}
```

```
window.addEventListener("load",start,false);
```

```
</script>
```

```
</head>
```

```
<body>
```

```
</body>
```

```
</html>
```

OK

Event Propagation-Event propagation means travelling of an element

Event Capturing

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Event Bubbling</title>
<script>
```

```
function parentmethod( e )
```

```
{
alert( "parent" );
}
```

```
function childmethod( e )
```

```
{
alert( "child" );}
```

```
function registerEvents()
```

```
{
document.getElementById("parent").addEventListener("click",
parentmethod, true );
document.getElementById("child").addEventListener("click",
childmethod, true );
}
```

```
window.addEventListener( "load", registerEvents, true );
```

```
</script>
```

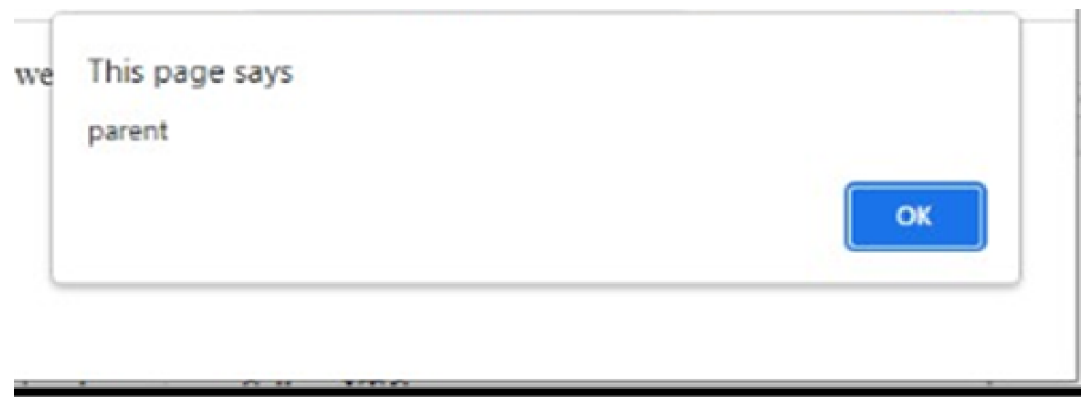
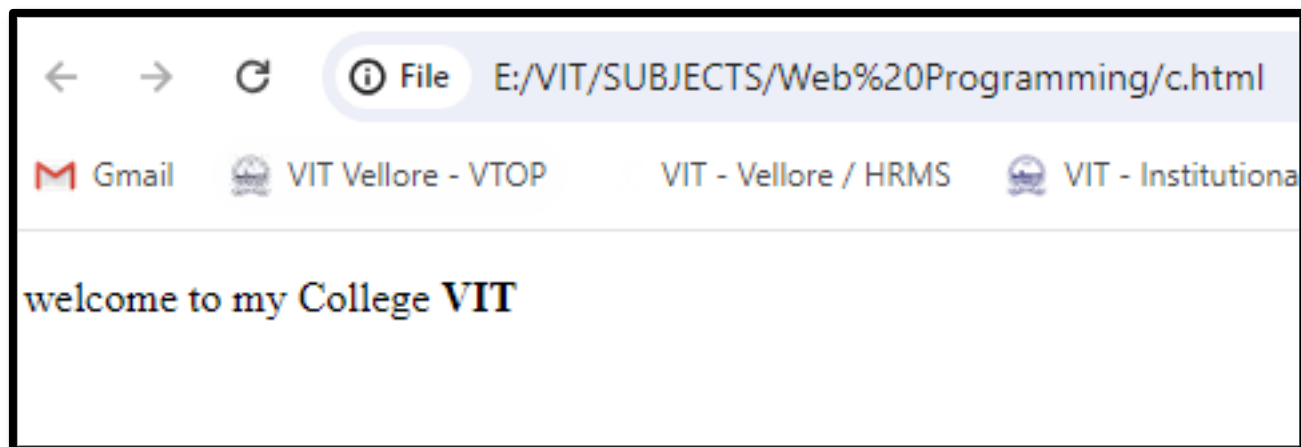
```
</head>
```

```
<body>
```

```
<p id = "parent">welcome to my College
```

```
<strong id = "child">KEC</strong></p>
```

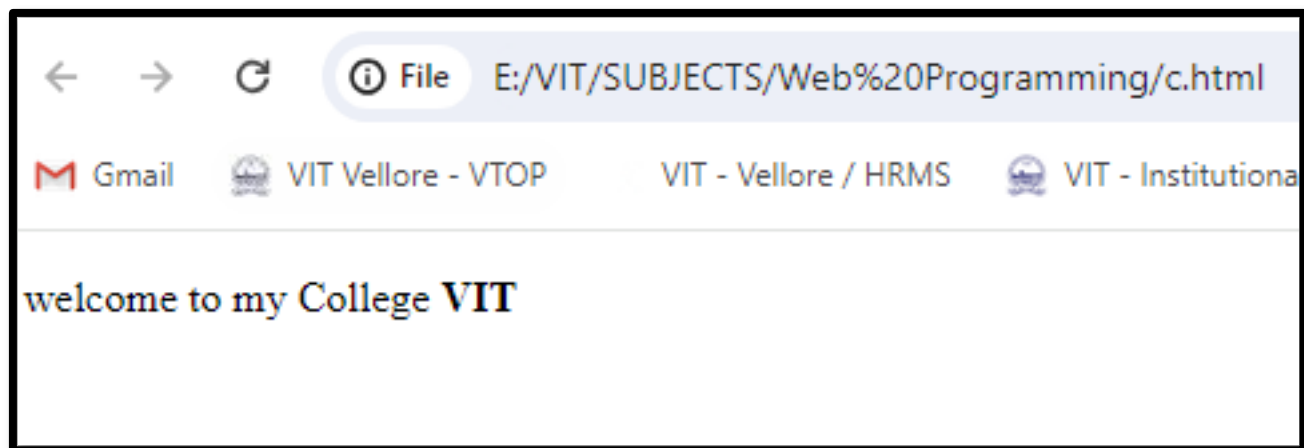
```
</body> </html>
```



Event Bubbling

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Event Bubbling</title>
<script>
function parentmethod( e )
{
alert( "parent" );
}
function childmethod( e )
{
alert( "child" );
}
```

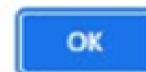
```
function registerEvents()
{
document.getElementById("parent").addEventListener("click",
parentmethod, false);
document.getElementById("child").addEventListener("click",
childmethod, false);
}
window.addEventListener( "load", registerEvents, false );
</script>
</head>
<body>
<p id = "parent">welcome to my College
<strong id = "child">KEC</strong></p>
</body>
</html>
```



we This page says
child



we This page says
parent



Stop Event Propagation→a handler can also take decision to stop the capturing when the event has been processed completely. In JavaScript, we use the **event.stopPropagation ()** method

Event Capturing

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="utf-8">
```

```
<title>Event Bubbling</title>
```

```
<script>
```

```
function parentmethod( e )
```

```
{
```

```
alert( "parent" );
```

```
}
```

```
function childmethod( e )
```

```
{
```

```
alert( "child" );}
```

```
function registerEvents()
```

```
{
```

```
document.getElementById( "parent").addEventListener( "click", parentmethod  
true);
```

```
document.getElementById("child").addEventListener("click",childmethod,  
true);
```

```
}
```

```
window.addEventListener( "load", registerEvents, true);
```

```
</script>
```

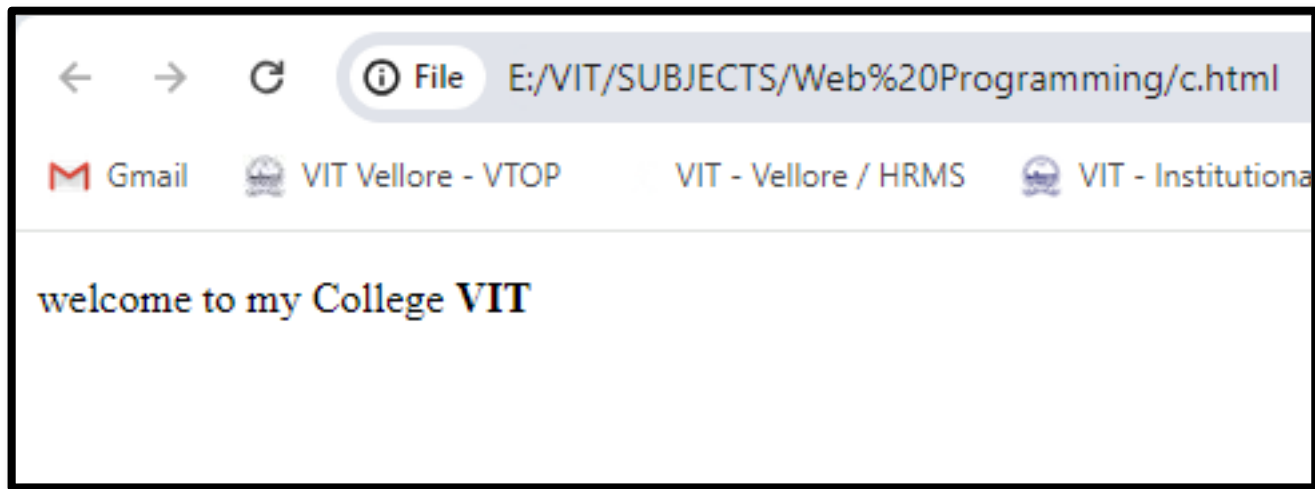
```
</head>
```

```
<body>
```

```
<p id = "parent">welcome to my College
```

```
<strong id = "child"onclick="event.stopPropagation()">VIT</strong></p>
```

```
</body> </html>
```



Stop Event Propagation→a handler can also take decision to stop the bubbling when the event has been processed completely. In JavaScript, we use the **event.stopPropagation ()** method

Event bubbling

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="utf-8">
```

```
<title>Event Bubbling</title>
```

```
<script>
```

```
function parentmethod( e )
```

```
{
```

```
  alert( "parent" );
```

```
}
```

```
function childmethod( e )
```

```
{
```

```
  alert( "child" );}
```

```
function registerEvents()
```

```
{
```

```
  document.getElementById("parent").addEventListener("click",parentmethod,  
  false);
```

```
  document.getElementById("child").addEventListener("click",childmethod,  
  false);
```

```
}
```

```
  window.addEventListener( "load", registerEvents, false);
```

```
</script>
```

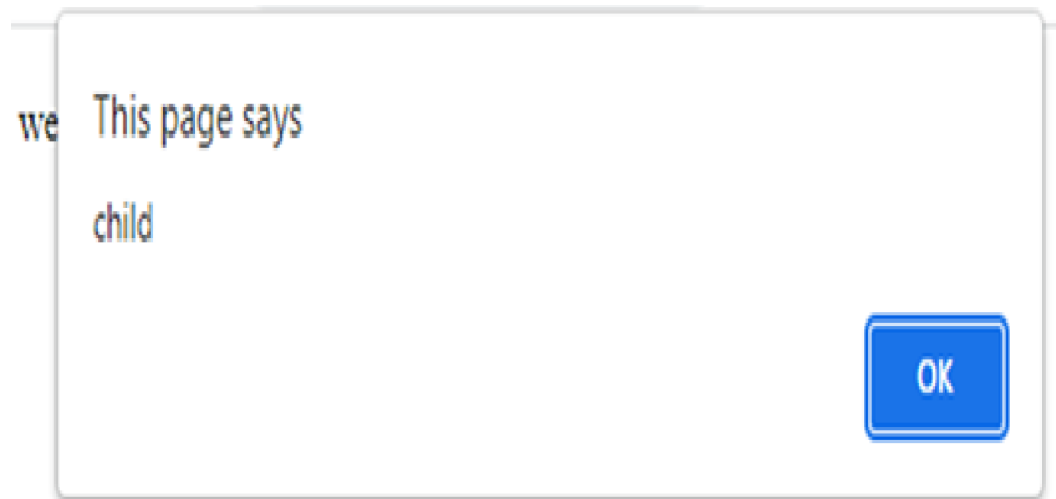
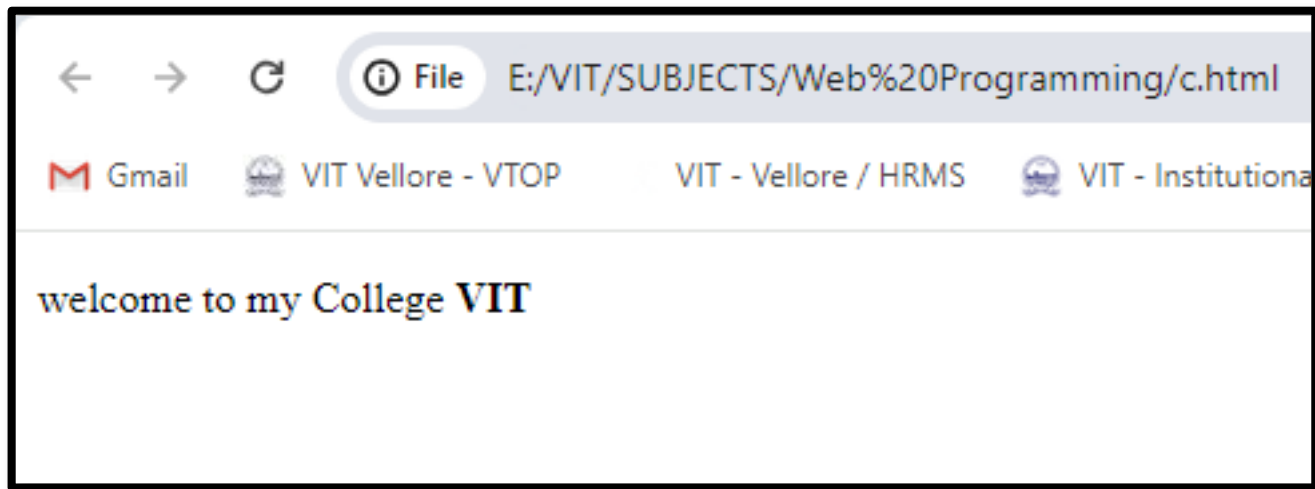
```
</head>
```

```
<body>
```

```
<p id = "parent">welcome to my College
```

```
<strong id = "child" onclick="event.stopPropagation()">VIT</strong></p>
```

```
</body> </html>
```



form Element

- form element, which is in charge of grouping a form's controls.
- Here's a template for the form element's syntax:

<form>

label

text-box, list-box, check-box, etc.

label

text-box, list-box, check-box, etc.

...

submit-button

</form>

Controls

input Element
button
text control
number
radio button
checkbox
password
date
color

select Element
pull-down menu
list box

textarea Element
textarea control

Some of the more popular controls and the elements used to implement them

Text Control

- `<input type="text" id="ssn" placeholder="#####" size="9" maxlength="9">`

Attributes

Here are the text control attributes we'll talk about in this subsection:

Text Control Attributes								
type	id	placeholder	size	maxlength	value	autofocus	disabled	readonly

Example:

```
<html>
```

```
<body><form>
```

First name:

```
<input type="text" id="fname" name="fname" autofocus><br><br>
```

Last name:

```
<input type="text" id="lname" name="lname"><br><br>
```

Last name:

```
<input type="text" id="lname" name="lname" disabled><br><br>
```

Country:

```
<input type="text" id="country" name="country" value="Norway" readonly><br><br>
```

```
<input type="submit">
```

```
</form>
```

```
</body></html>
```

First name:

Last name:

Last name:

Country:

Submit

Accessing a Form's Control Values

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="author" content="John Dean">
<title>Email Address Generator</title>
<script>
function generateEmail(form) {
document.getElementById("email").innerHTML =
form.elements["first"].value + "." +
form.elements["last"].value + "@gmail.com";
form.reset();
form.elements["first"].focus();
} </script>
</head>
```

```
<body><h3>
```

Enter your first and last names and then click the button.

```
</h3>
```

```
<form>
```

First Name:

```
<input type="text" id="first" size="15" autofocus><br>
```

Last Name:

```
<input type="text" id="last" size="15"><br><br>
```

```
<input type="button" value="Generate Email" onclick="generateEmail(this.form);">
```

```
<p id="email"></p>
```

```
</body></form></html>
```

Enter your first and last names and then click the button.

First Name:

Last Name:

Enter your first and last names and then click the button.

First Name:

Last Name:

John.Peter@gmail.com

Form events:

Submit and Reset

- **submit and reset** events fire when a form is submitted or reset

Focus and Blur

- useful when dealing with form elements that allow user input
- **focus event** fires when an element gains the focus (i.e., when the user clicks a form field or uses the Tab key to move between form elements)
- **blur fires** when an element loses the focus, which occurs when another control gains the focus.

Example(form)

```
<html><head>
<title> form events </title>
<script>
window.addEventListener("load", start,false);
function start()
{
document.getElementById("fname").addEventListener("focus",focusone,false);
document.getElementById("fname").addEventListener("blur",blurone,false);
document.getElementById("myform").addEventListener("submit",subfun,false);
document.getElementById("myform").addEventListener("reset",resetfun,false);
}
function focusone()
{
document.getElementById("help").innerHTML="Enter Name";}
```

```
function blurone()
{
document.getElementById("help").innerHTML=" ";
}

function subfun()
{
window.alert("Are you sure to submit");
window.alert("Thank you");
}

function resetfun()
{
window.alert("Are you sure to reset");
}

</script>
</head>
```

```
<body>
```

```
<form id="myform">
```

```
Enter your name: <input type="text" id="fname"><br>
```

```
Enter Rollno:<input type="text1" id="roll"><br>
```

```
<input type="submit" id="sub"><br>
```

```
<input type="reset" id="roll"><br>
```

```
</form>
```

```
<p id="help"></p>
```

```
</body>
```

```
</html>
```

Enter your name:

Enter Rollno:

Submit

Reset

Enter your name:

Enter Rollno:

Submit

Reset

Enter Name

En This page says

En Are you sure to submit

S

OK

Reset

En This page says

En Are you sure to reset

S

OK

Reset

Event Handler Attributes

OnChange: The OnChange event occurs when the value of an HTML element is changed.

Mouse events:

mousemove - The event occurs when the pointer is moving while it is over an element

mouseover - The event occurs when the pointer is moved onto an element

mouseout - The event occurs when a user moves the mouse pointer out of an element

Example : OnChange

```
<html>
```

```
<body>
```

```
<form>
```

```
Enter your name: <input type="text" id="fname"></form>
```

```
<p>When you leave the input field, a function is triggered which transforms the input text to upper case.</p>
```

```
<script>
```

```
document.getElementById("fname").addEventListener("change", myFunction);
```

```
function myFunction() {
```

```
    var x = document.getElementById("fname");
```

```
    x.value = x.value.toUpperCase();
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Enter your name:

When you leave the input field, a function is triggered which transforms the input text to upper case.

Enter your name:

When you leave the input field, a function is triggered which transforms the input text to upper case.

Enter your name:

When you leave the input field, a function is triggered which transforms the input text to upper case.

Example(mousemove and mouseout)

```
<html>
<head>
<title> mouse events </title>
</head>
<body>
<h1 id="move">My college and My department </h1>
<script>
document.getElementById("move").addEventListener("mousemove",moveone,false);
document.getElementById("move").addEventListener("mouseout",leaveone,false);
function moveone()
{
document.getElementById("move").innerHTML= "Vellore Institute of Technology";
}
function leaveone()
{
document.getElementById("move").innerHTML="Computer Science and Engineering";
}
</script></body></html>
```



Event Target

- The target event property returns the element that triggered the event.
- The target property can be the element that registered for the event
- It is necessary to have the target property when an event is fired. We can do the following with the target property of the event.
 - Get the element that fired the event.
 - Access the properties of the element.

Example(mousemove and mouseout with event target)

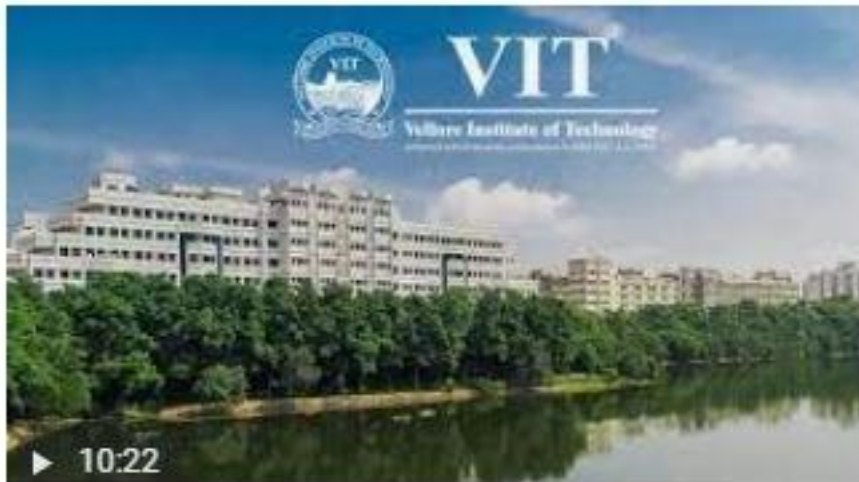
```
<html>
<head>
<title> mouse events </title>
</head>
<body>
<h1 id="move">My college and My department </h1>
<script>
document.getElementById("move").addEventListener("mousemove",moveone,false);
document.getElementById("move").addEventListener("mouseout",leaveone,false);
function moveone(e)
{
e.target.innerHTML="Vellore Institute of Technology";
}
function leaveone()
{
document.getElementById("move").innerHTML="Computer Science and Engineering";
}
</script></body></html>
```



Example(mouseover and mouseout)

```
<html>
<head>
<title> mouse events </title>
<script>
window.addEventListener("load", start,false);
function start()
{
document.getElementById("picture").addEventListener("mouseover",over,false);
document.getElementById("picture").addEventListener("mouseout",out,false);
}
function over(e)
{
e.target.setAttribute("src","rose.jpg");
}
function out(e)
{
e.target.setAttribute("src","taj.jpg");
}
</script></head>
```

```
<body>  
  
</body>  
</html>
```



Manipulating CSS with JavaScript-

1. Assigning a New Value to an Element's class Attribute

```
<html>
```

```
<head><style>
```

```
.italic {font-style: italic;}
```

```
.bold {font-weight: bold;}
```

```
.small-caps {font-variant: small-caps;}
```

```
</style></head>
```

```
<body>
```

```
<div class="italic" id="message">
```

CSS is the acronym of "Cascading Style Sheets". It is used for laying out and structuring web pages</div>

```
<button onClick="change()">Change Style </button>
```

```
<script>
```

```
function change()
```

```
{
```

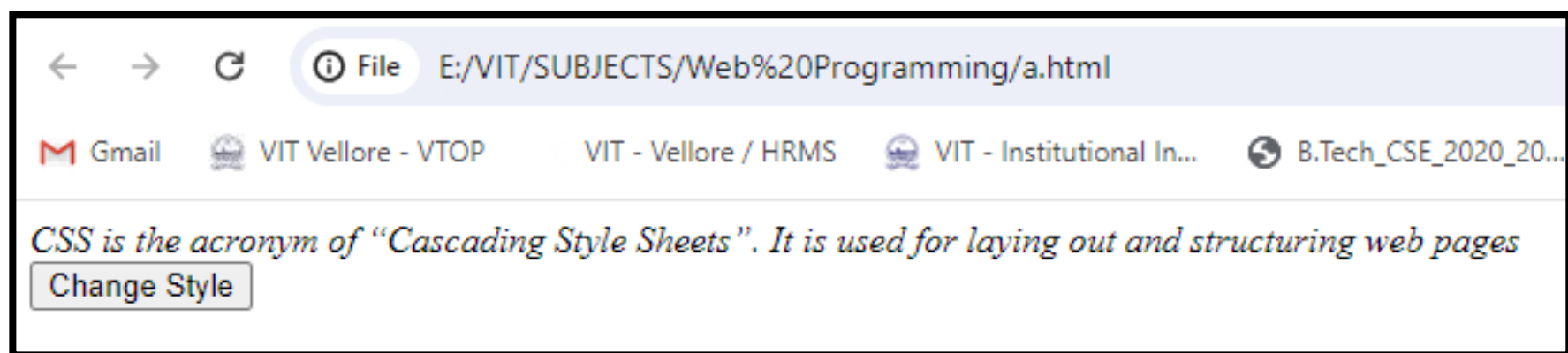
```
document.getElementById("message").className =  
"bold small-caps";
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```



Manipulating CSS with JavaScript-

2. Assigning a Value to an Element's style Property

```
<html><body>
```

```
<div id="message">
```

CSS is the acronym of "Cascading Style Sheets". It is used for laying out and structuring web pages

```
</div>
```

```
<button onClick="change()">Change Style </button>
```

```
<script>
```

```
function change()
```

```
{
```

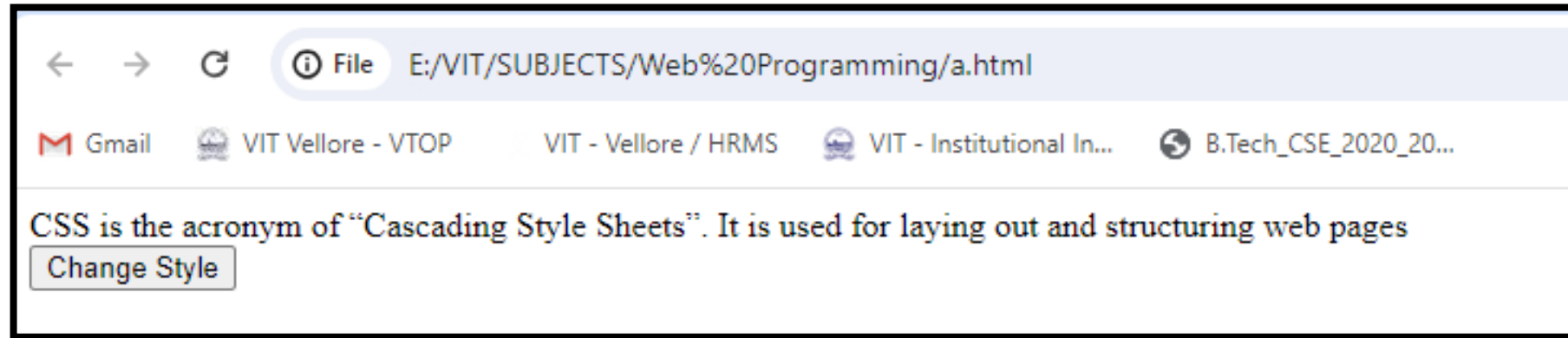
```
document.getElementById("message").style.fontSize = "3em";
```

```
document.getElementById("message").style.color = "red";
```

```
}
```

```
</script></body>
```

```
</html>
```



Using z-index to Stack Elements

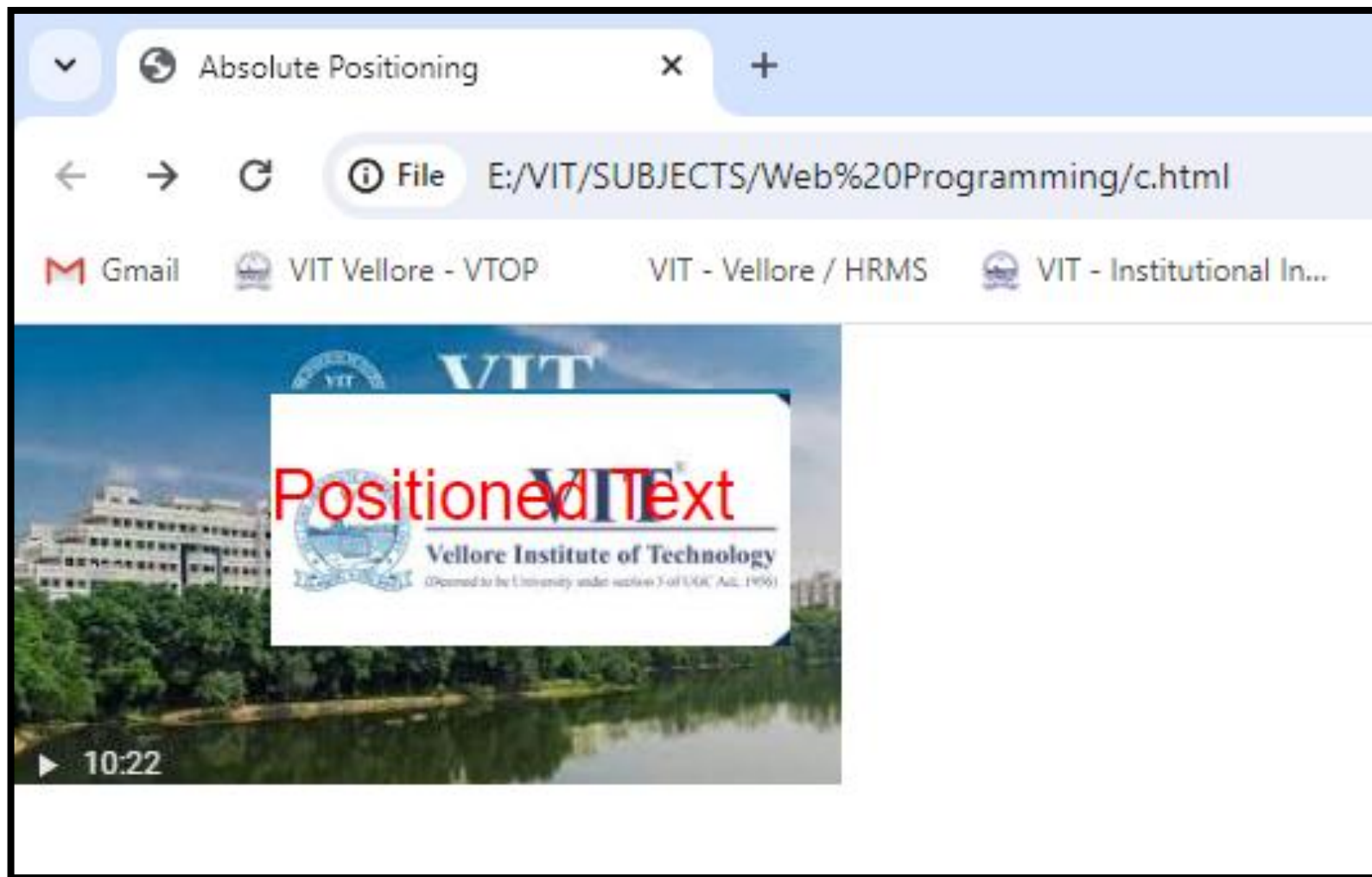
z-index Property

- The **z-index** property specifies the stack order of an element.
- An element with greater stack order is always in front of an element with a lower stack order

```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>Absolute Positioning</title>
<style type = "text/css">
.image1
{
position: absolute;
top: 0px;
left: 0px;
z-index:1;
}
.image2
{
position: absolute;
top: 25px;
left: 100px;
z-index:2;
}
```

```
.text
{
position: absolute;
top: 25px;
left: 100px;
z-index:3;
font-size: 20pt;
font-family:Arial;
color:red;
}
</style>
</head>
<body>


<p class = "text">Positioned Text</p>
</body>
</html>
```



1. Using JavaScript to Modify the Stacking Order

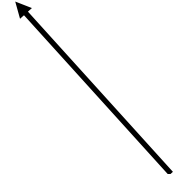
```
<!DOCTYPE html>
<html>
<head>
<meta charset = "utf-8">
<title>Absolute Positioning</title>
<style type = "text/css">
.image1
{
position: absolute;
top: 0px;
left: 0px;
z-index:1;
}
.image2
{
position: absolute;
top: 25px;
left: 100px;
z-index:2;
}
```

```
.text
{
position: absolute;
top: 25px;
left: 100px;
z-index:3;
font-size: 20pt;
font-family:Arial;
color:red;
}</style></head>
<body>

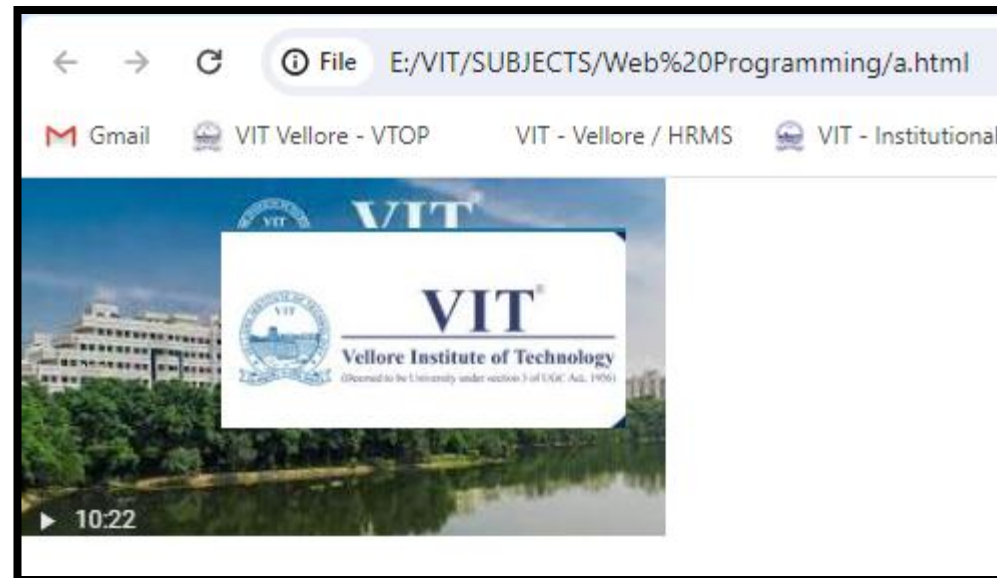
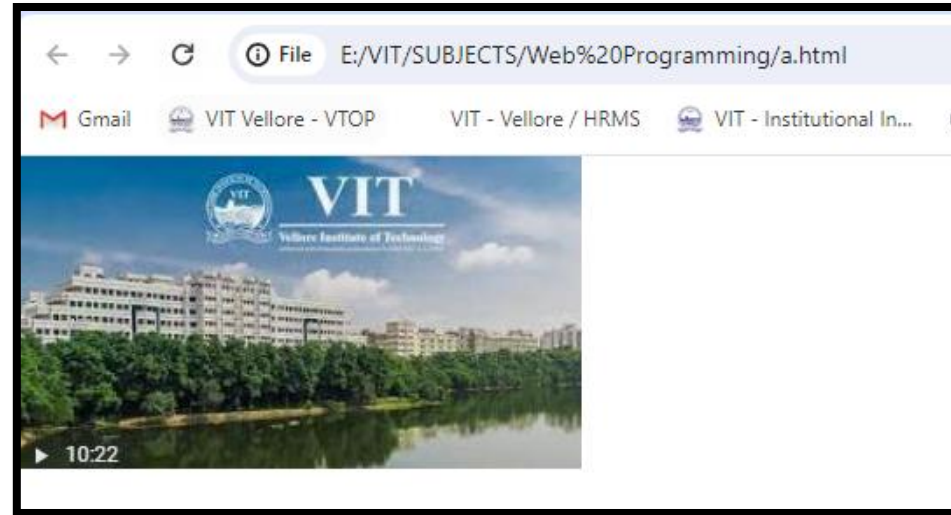
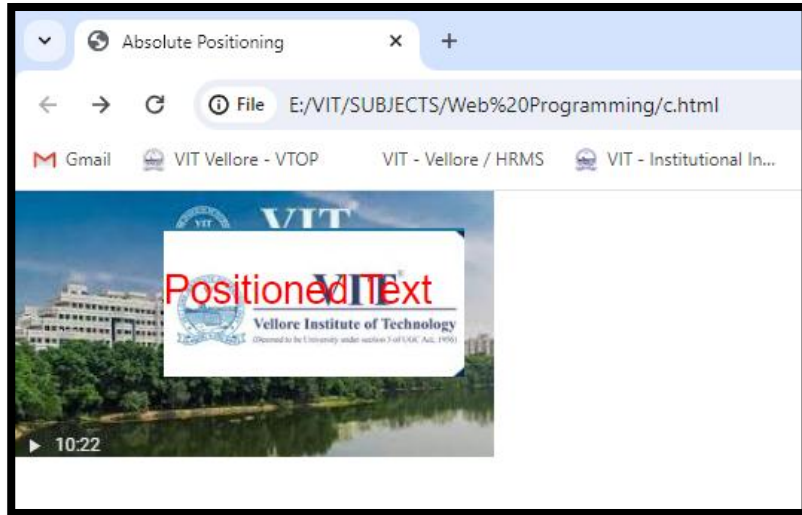

<p class = "text">Positioned Text</p>
<script>
var topIndex = 3;
function moveToTop(picture){
picture.style.zIndex = ++topIndex;
}</script>
</body></html>
```

Note:

this refers to the clicked image's object



The picture parameter receives the clicked image's object.



Canvas and Drawing

- To use canvas,
 - (1) a **canvas element** and
 - (2) JavaScript method calls that draw graphics objects within the canvas element's drawing area.
- The **canvas element is used to draw graphics**, on the fly, via JavaScript.
- The **canvas element is only a container for graphics**. use JavaScript to actually draw the graphics.
- Canvas has several methods for drawing paths, boxes, circles, text, and adding images.

Empty canvas

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

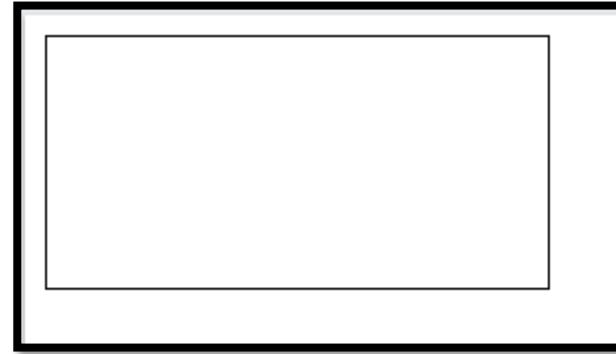
```
<canvas id="myCanvas" width="200" height="100" style="border:1px  
solid #000000;">
```

Your browser does not support the HTML canvas tag.

```
</canvas>
```

```
</body>
```

```
</html>
```



Rectangles Web Page

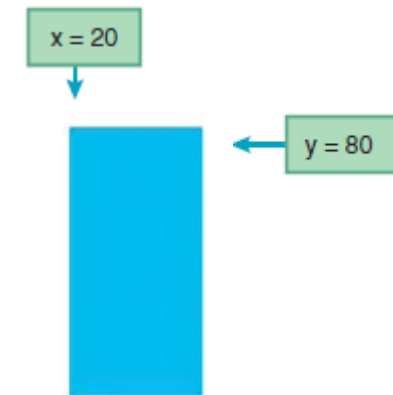
- To draw a filled-in rectangle, use the context object to call the **fillRect** method. Here's the syntax: `context.fillRect(x, y, width, height);`

- **Fill Color →**

- `ctx.fillStyle = "deepskyblue";`
- `ctx.fillRect(20, 80, 70, 140);`

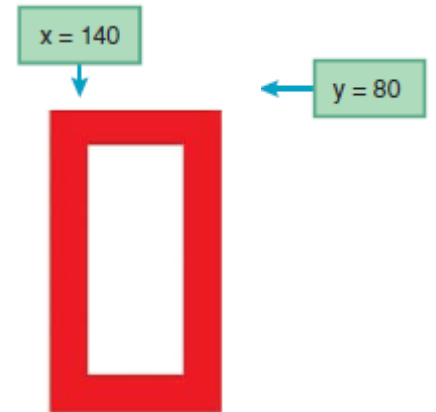
- **color value formats→**

- color name
- RGB value—specifies amounts of red, green, and blue
- RGBA value—specifies RGB, plus the amount of opacity
- HSL value—specifies amounts of hue, saturation, and lightness
- HSLA—specifies HSL, plus the amount of opacity



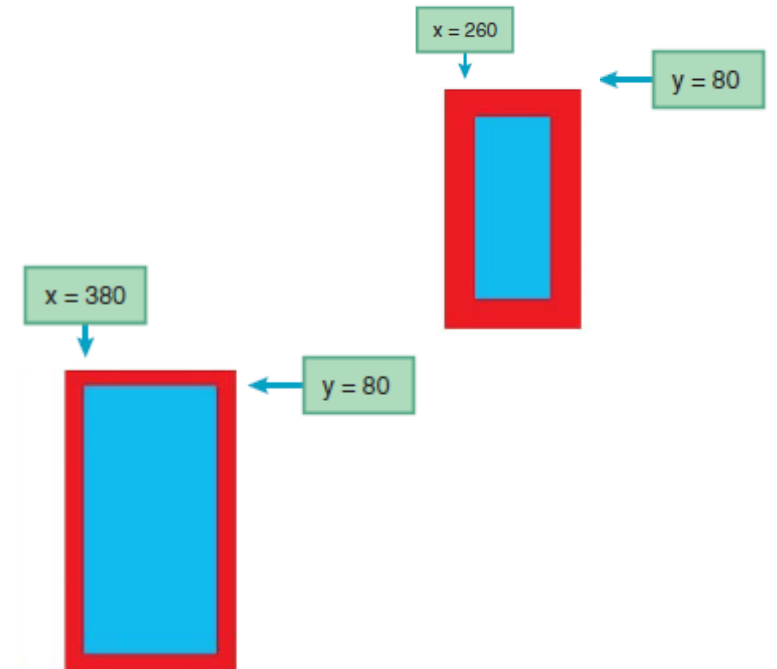
- Rectangle Borders→

- `context.strokeRect(x, y, width, height);`
- `ctx.lineWidth = 20;` (default width is 1 pixel)
- `ctx.strokeStyle = "red";`(default drawing color is black)



- A rectangle, which has different colors for its border and interior

- Case 1: `ctx.fillRect(260, 80, 70, 140);`
`ctx.strokeRect(260, 80, 70, 140);`
- Case 2: `ctx.strokeRect(260, 80, 70, 140);`
`ctx.fillRect(260, 80, 70, 140);`



Example:

```
<!DOCTYPE html>

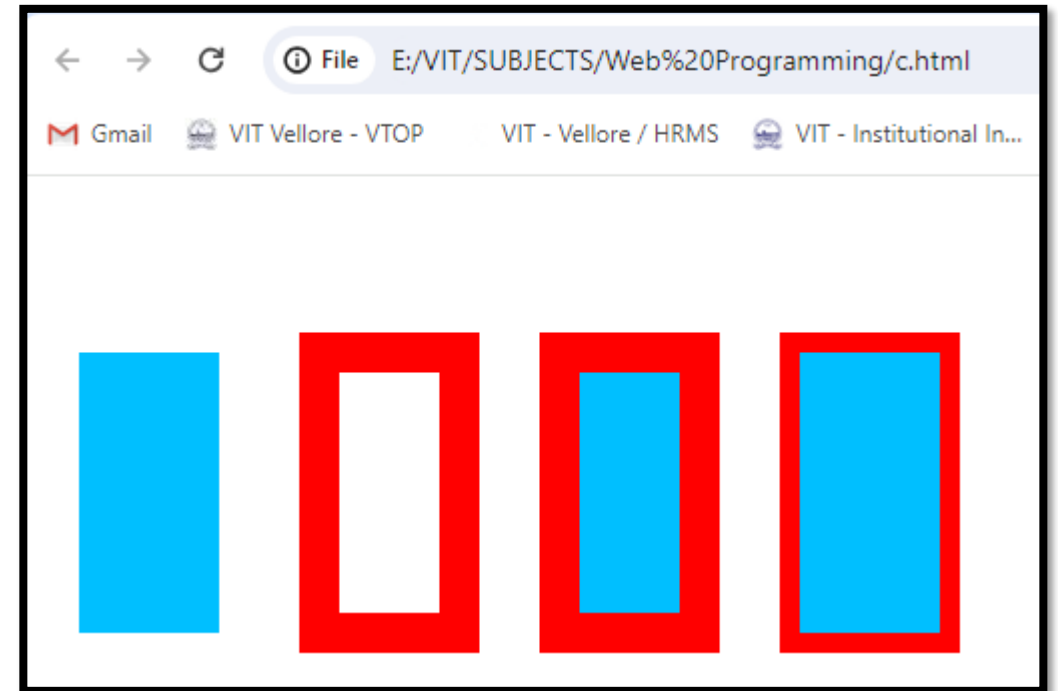
<head>

<title>Rectangle Examples</title>

<script>

function rectangleExamples() {
var ctx; // the canvas object's context
ctx = document.getElementById("canvas").getContext("2d");
ctx.fillStyle = "deepskyblue";
ctx.fillRect(20, 80, 70, 140);
ctx.lineWidth = 20;
ctx.strokeStyle = "red";
ctx.strokeRect(140, 80, 70, 140);
ctx.fillRect(260, 80, 70, 140);
ctx.strokeRect(260, 80, 70, 140);
```

```
ctx.strokeRect(380, 80, 70, 140);  
ctx.fillRect(380, 80, 70, 140);  
} // end rectangleExamples  
</script>  
</head>  
<body onload="rectangleExamples();">  
<canvas id="canvas" width="480" height="250">  
Sorry - This page uses <code>canvas</code>, and  
your browser doesn't support it.  
</canvas>  
</body>  
</html>
```



Using a Color Gradient to Fill a Shape

- Syntax for retrieving a gradient object from the context object

`context.createLinearGradient(x1, y1, x2, y2);`

- The `x1, y1` arguments form the coordinates for the point where you can specify the `first color` in your gradient, and the `x2, y2` arguments form the coordinates for the point where you can specify the `last color` in your gradient
- To specify the colors, call the gradient object's `addColorStop` method one time for each color.

Here's the syntax: `gradient.addColorStop(offset, color);`

- The `offset` argument, a number `between 0 and 1`, specifies where the color is positioned within the gradient's range. A `0 value` indicates that the color is at the starting point formed by `x1, y1`. A `1 value` indicates that the color is at the ending point formed by `x2`

Example:

```
<!DOCTYPE html>
```

```
<head>
```

```
<title>Rectangle Examples</title>
```

```
<script>
```

```
function rectangleExamples() {
```

```
    var ctx; // the canvas object's context
```

```
    ctx = document.getElementById("canvas").getContext("2d");
```

```
    var gradient = ctx.createLinearGradient(0, 80, 0, 220); // Adjust the gradient coordinates
```

```
    gradient.addColorStop(0, "green");
```

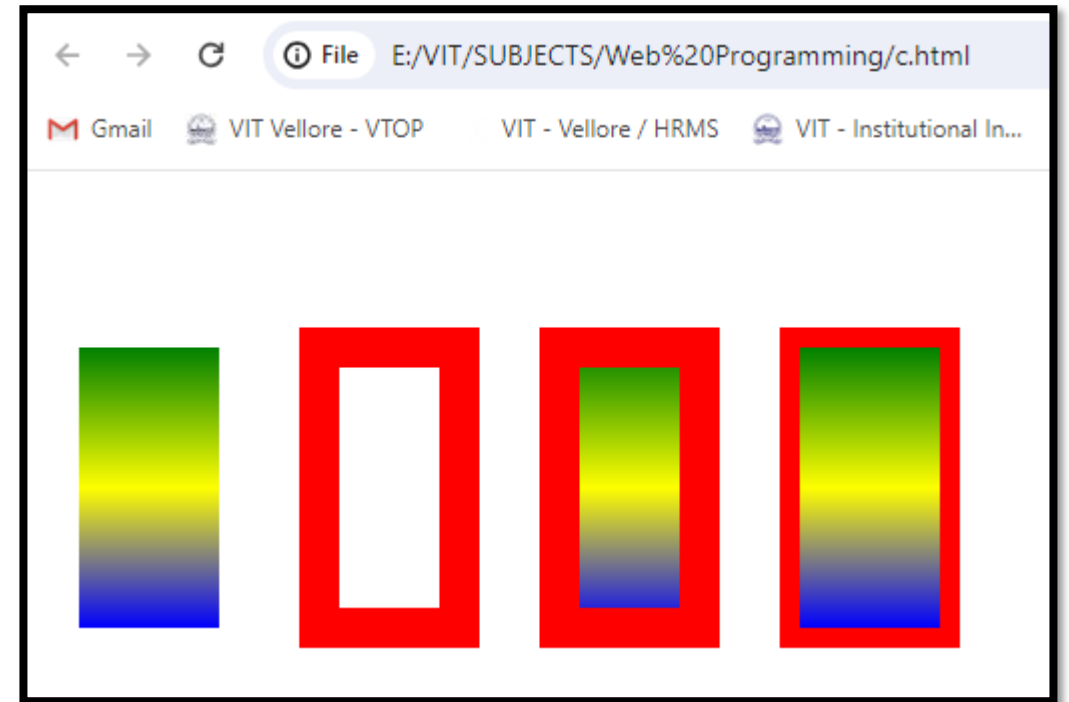
```
    gradient.addColorStop(0.5, "yellow");
```

```
    gradient.addColorStop(1, "blue");
```

```
    ctx.fillStyle = gradient;
```

```
    ctx.fillRect(20, 80, 70, 140);
```

```
ctx.lineWidth = 20;
ctx.strokeStyle = "red";
ctx.strokeRect(140, 80, 70, 140);
ctx.fillStyle = gradient; // Reuse the same gradient
ctx.fillRect(260, 80, 70, 140);
ctx.strokeRect(260, 80, 70, 140);
ctx.strokeRect(380, 80, 70, 140);
ctx.fillRect(380, 80, 70, 140);
} // end rectangleExamples
</script>
</head>
<body onload="rectangleExamples();">
  <canvas id="canvas" width="480" height="250">
    Sorry - This page uses <code>canvas</code>, and your browser doesn't support it. </canvas>
  </body></html>
```



Drawing Text with fillText and strokeText

- To draw text character interiors, use the context object's fillText method.

`context.fillText(text, x, y, max-width);`

- The x and y parameters are coordinates for the placement of the lower-left corner of the text string within the canvas drawing area. The max-width parameter is optional, and it specifies the maximum width of the text argument's string.

- The strokeText method works the same as the fillText method except that it draws its text argument characters with borders, and it leaves the characters' interiors empty.

`context.strokeText(text, x, y, max-width);`

Formatting Text

font Property

- `context.font = [font-style-value] [font-variant-value] [font-weight-value] font-size-value[/line-height-value] font-family-value;`
- Ex: `ctx.font = "2em 'Times New Roman', serif";`

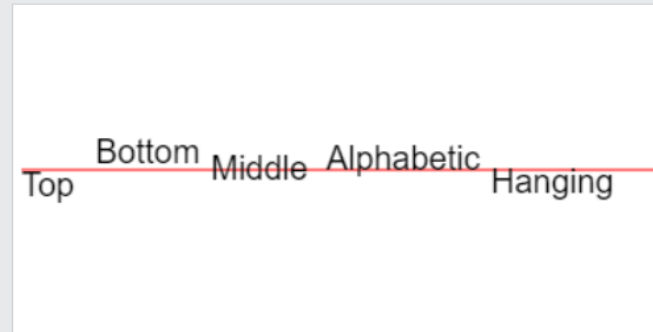
textAlign property

- `ctx.textAlign = "center";`

textBaseline Property

- This property sets or returns the text baseline used when drawing text.
- Default value is alphabetic

Draw a red line at y=100, then place each word at y=100 with different textBaseline values:



Example:

```
<!DOCTYPE html>

<head>

<title>George Orwell's 1984</title>

<script>

function displayQuotes() {
var ctx; // the canvas object's context
ctx = document.getElementById("canvas").getContext("2d");
ctx.font = "2em 'Times New Roman', serif";
ctx.fillText("\nThe clocks were striking thirteen.\'", 5, 30);
ctx.textAlign = "center";
ctx.textBaseline = "top";
ctx.fillText("\nThe clocks were striking thirteen.\'", 300, 110, 350);
ctx.strokeText("\'He loved Big Brother.\'", 300, 160, 350);
} </script>
```

```
</head>
```

```
<body onload="displayQuotes();">
```

```
<canvas id="canvas" width="600" height="170">
```

```
</canvas>
```

```
</body>
```

```
</html>
```



Drawing Arcs and Circles

False → clockwise direction (default)
True → Counterclockwise

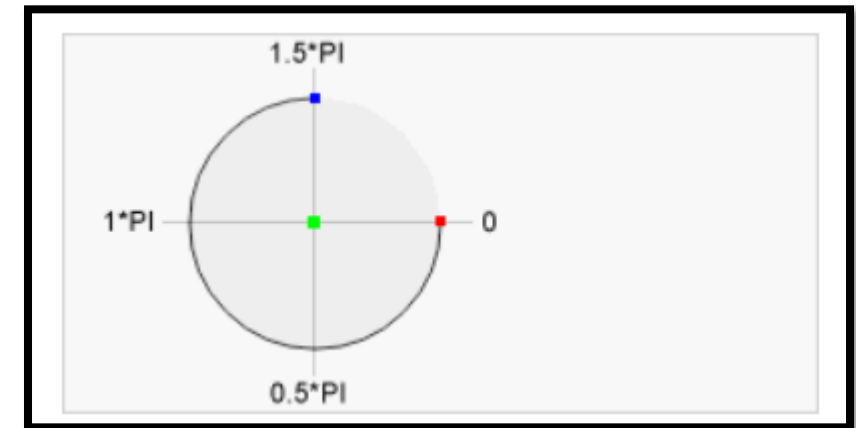
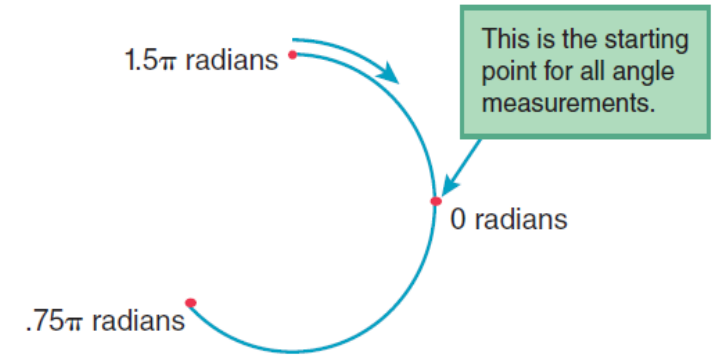
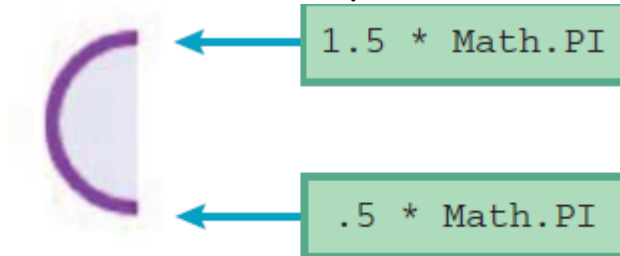
- `context.arc(x, y, radius, start-angle, end-angle, clockwise);`

- `context.arc(x, y, radius, 1.5 * Math.PI, .75 * Math.PI);`

- `ctx.arc(80, 35, 25, 0, 2 * Math.PI); // circle creation`



- `ctx.arc(80, 35, 25, .5 * Math.PI, 1.5 * Math.PI);`



Example:

```
<!DOCTYPE html>
```

```
<head>
```

```
<script>
```

```
function displayQuotes() {
```

```
    var ctx; // the canvas object's context
```

```
    ctx = document.getElementById("canvas").getContext("2d");
```

```
    ctx.strokeStyle = "darkviolet";
```

```
    ctx.fillStyle = "lavender";
```

```
    ctx.lineWidth = 4;
```

```
    // Draw a circle
```

```
    ctx.beginPath();
```

```
    ctx.arc(80, 80, 25, 0, 2 * Math.PI);
```

```
    ctx.fill();
```

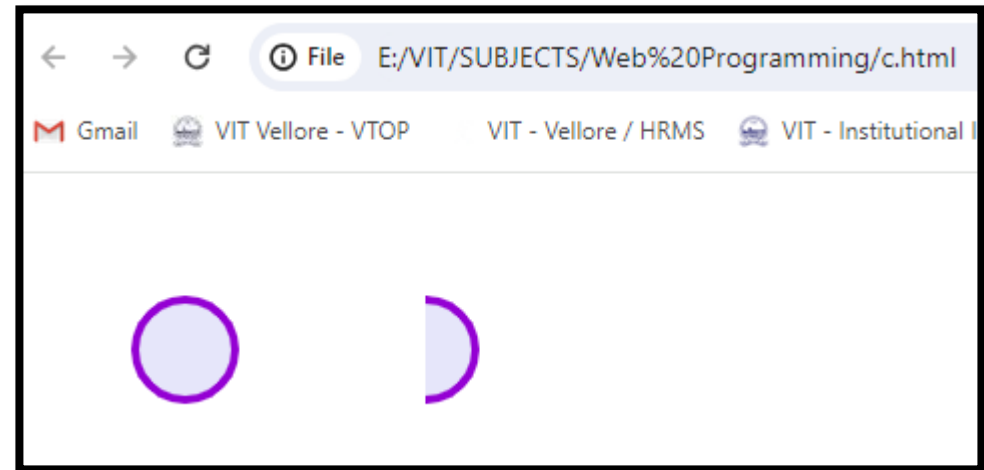
```
    ctx.stroke();
```

// Draw an arc in Counterclockwise direction

False → clockwise direction
(default)

True → Counterclockwise

```
ctx.beginPath();
ctx.arc(200, 80, 25, .5 * Math.PI, 1.5 * Math.PI, true);
ctx.fill();
ctx.stroke();
} // end displayMsg
</script>
</head>
<body onload="displayQuotes();">
<canvas id="canvas" width="600" height="200">
</canvas>
</body>
</html>
```



Drawing Lines and Paths

- *moveTo method(it moves the drawing pen)*

```
context.moveTo(starting-x, starting-y);
```

- *lineTo method(it moves the pen to the line's ending position)*

```
context.lineTo(ending-x, ending-y);
```

- *beginPath()(creating a path)*

```
context.beginPath();
```

- *stroke(draws the current path)*

```
context.stroke();
```

Example:

```
<!DOCTYPE html><head><script>
```

```
function displayQuotes() {
```

```
    var ctx; // the canvas object's context
```

```
    ctx = document.getElementById("canvas").getContext("2d");
```

```
    ctx.strokeStyle = "red";
```

```
    ctx.lineWidth = 4;
```

```
    ctx.beginPath();
```

```
    ctx.moveTo(40, 20);
```

```
    ctx.lineTo(80, 20);
```

```
    ctx.moveTo(90, 20);
```

```
    ctx.lineTo(130, 20);
```

```
    ctx.moveTo(140, 20);
```

```
    ctx.lineTo(180, 20);
```

```
    ctx.stroke();
```

```
}</script>
```

```
</head>
```

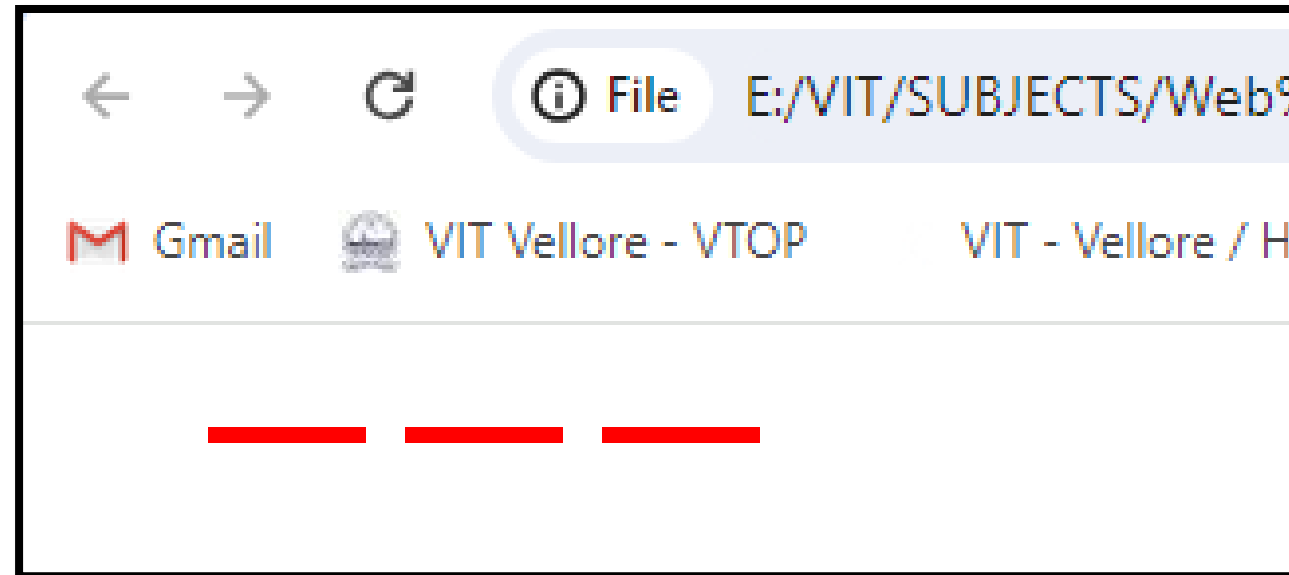
```
<body onload="displayQuotes();">
```

```
<canvas id="canvas" width="600" height="200">
```

```
</canvas>
```

```
</body>
```

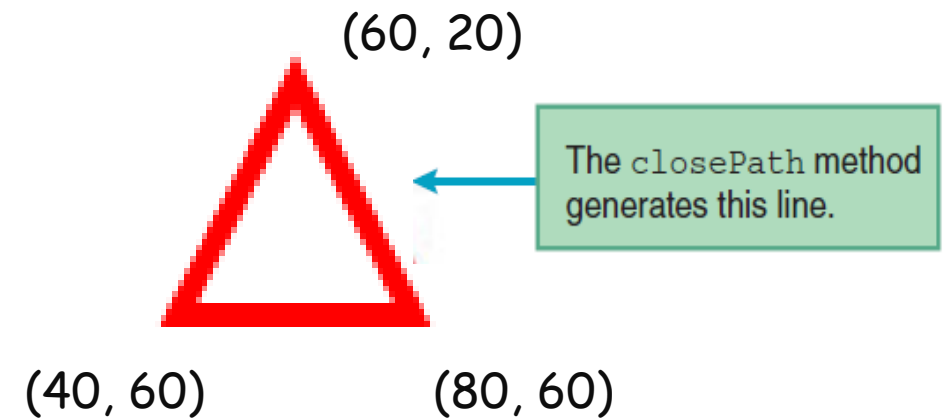
```
</html>
```



Closing a Path

- *closePath method connects the most recently created line's end point to the most recently created subpath's starting point.*

- `ctx.moveTo(60, 20);`
- `ctx.lineTo(40, 60);`
- `ctx.lineTo(80, 60);`
- `ctx.closePath();`



Connects the last point (80, 60) back to the starting point (60, 20), creating a closed shape. In this case, **it forms a triangle** with vertices at (60, 20), (40, 60), and (80, 60).

Example:

```
<!DOCTYPE html><head>
```

```
<script>
```

```
function displayQuotes() {
```

```
    var ctx; // the canvas object's context
```

```
    ctx = document.getElementById("canvas").getContext("2d");
```

```
    ctx.strokeStyle = "red";
```

```
    ctx.lineWidth = 4;
```

```
    ctx.beginPath();
```

```
    ctx.moveTo(60, 20);
```

```
    ctx.lineTo(40, 60);
```

```
    ctx.lineTo(80, 60);
```

```
    ctx.closePath();
```

```
    ctx.stroke();
```

```
}</script>
```

```
</head>
```

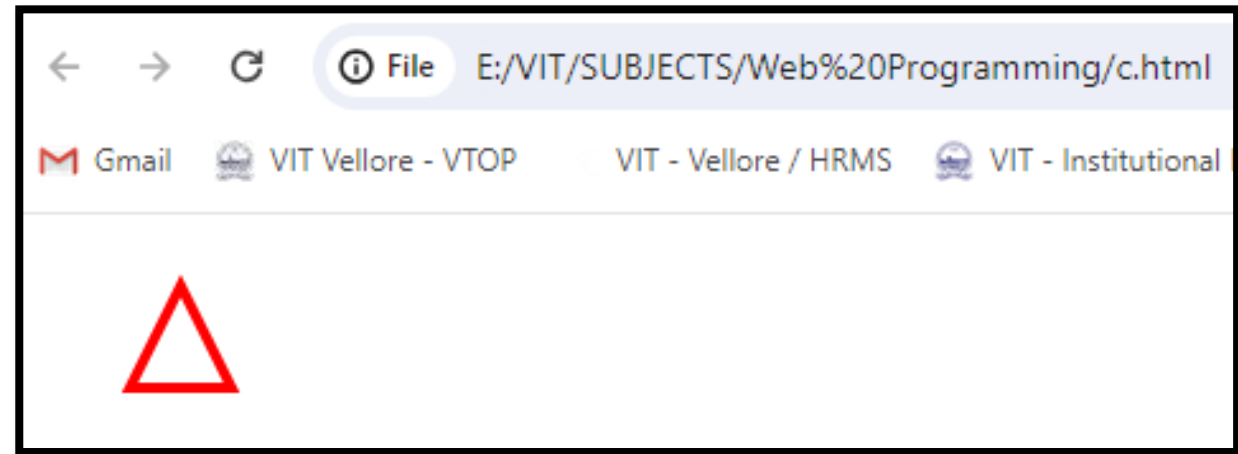
```
<body onload="displayQuotes();">
```

```
<canvas id="canvas" width="600" height="200">
```

```
</canvas>
```

```
</body>
```

```
</html>
```



Splitting a Path

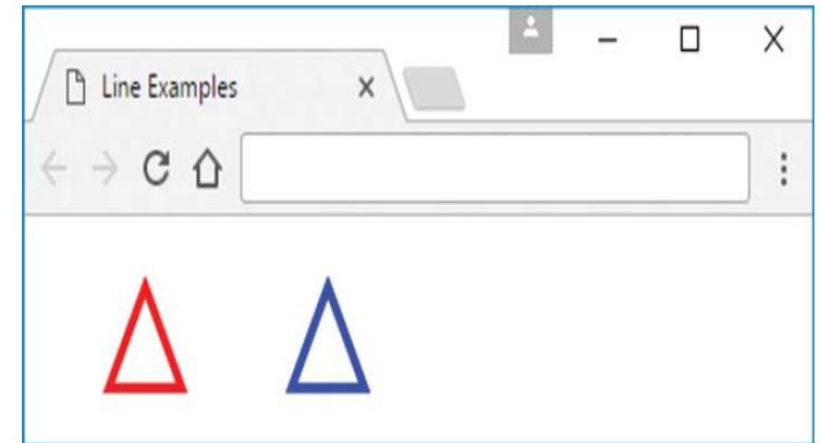
- if you want to change the line properties for a drawing, you need to **split the path into multiple paths**.
- And for each of the resulting smaller paths, you call `beginPath` at the start of the path and call `stroke` at the end of the path.

```
ctx.strokeStyle = "red";  
ctx.lineWidth = 4;  
ctx.beginPath();  
ctx.moveTo(60, 20);  
ctx.lineTo(40, 60);  
ctx.lineTo(80, 60);  
ctx.closePath();  
ctx.stroke();
```

```
ctx.strokeStyle = "blue";  
ctx.beginPath();  
ctx.moveTo(160, 20);  
ctx.lineTo(140, 60);  
ctx.lineTo(180, 60);  
ctx.closePath();  
ctx.stroke();
```

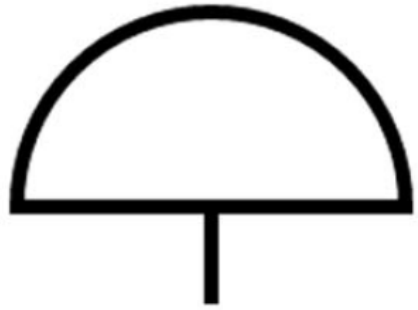


Split the path so the color can change.

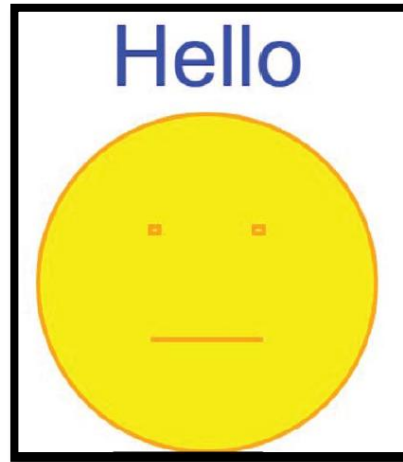


Practice Problems

- Simple Umbrella web page creation



- Face web page Creation



Using Canvas for Transformations

Translation:

context.translate(x, y);

The translate method moves the origin and the coordinate system x pixels to the right and y pixels down

Rotate:

context.rotate(clockwise-rotation-in-radians);

Rotation always takes place around the coordinate system's origin.

Scaling

context.scale(x-scale-factor, y-scale-factor);

This method scales the current context..

Example:

```
<!DOCTYPE html><head>
```

```
<script>
```

```
function displayQuotes() {
```

```
    var ctx; // the canvas object's context
```

```
    ctx = document.getElementById("canvas").getContext("2d");
```

```
    // Draw a square before transformations
```

```
    ctx.fillStyle = "blue";
```

```
    ctx.fillRect(50, 50, 50, 50);
```

```
    // Translation, rotation, scale
```

```
    ctx.translate(100, 100);
```

```
    ctx.rotate(20 * Math.PI / 180);
```

```
    ctx.scale(2, 2);
```

```
    ctx.fillStyle = "green";
```

```
    ctx.fillRect(50, 50, 50, 50);
```

```
}</script>
```

The **Math.PI / 180** part is where you do the converting to radians. You then **multiply by 20 degrees** you actually want.

```
</head>
```

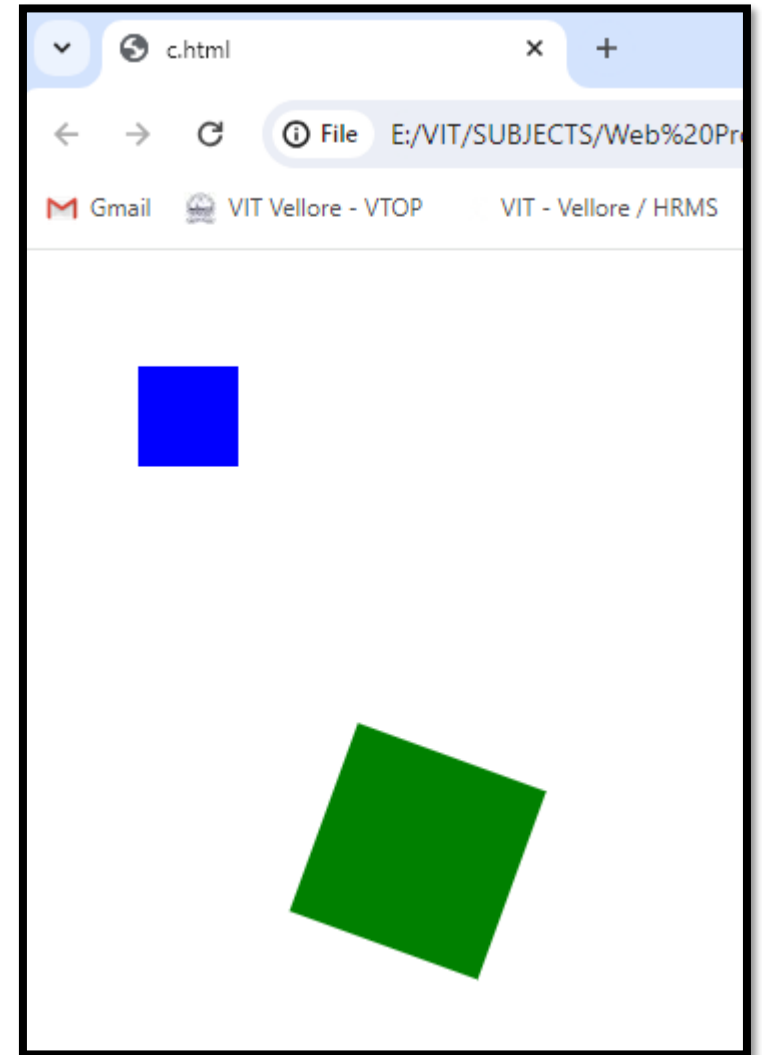
```
<body onload="displayQuotes();">
```

```
<canvas id="canvas" width="600" height="400">
```

```
</canvas>
```

```
</body>
```

```
</html>
```



transform()

- This method scales, rotates, moves, and skews the context.
- `ctx.transform(1, 0.5, -0.5, 1, 30, 10);`

1.Horizontal Scaling (m11): 1

- This parameter sets the horizontal scaling factor to **1**, meaning no horizontal scaling.

2.Horizontal Skewing (m21): 0.5

- This parameter represents horizontal skewing. It sets the horizontal skewing factor to **0.5**, which skews the content horizontally.

3.Vertical Skewing (m12): -0.5

- This parameter represents vertical skewing. It sets the vertical skewing factor to **-0.5**, which skews the content vertically.

4.Vertical Scaling (m22): 1

- This parameter sets the vertical scaling factor to **1**, meaning no vertical scaling.

5.Horizontal Translation (dx): 30

- This parameter translates the drawing horizontally by **30** units.

6.Vertical Translation (dy): 10

- This parameter translates the drawing vertically by **10** units.

Example:

```
<!DOCTYPE html><head>
```

```
<script>
```

```
function displayQuotes() {
```

```
    var ctx; // the canvas object's context
```

```
    ctx = document.getElementById("canvas").getContext("2d");
```

```
    // Draw a square before transformation
```

```
    ctx.fillStyle = 'blue';
```

```
    ctx.fillRect(50, 50, 50, 50);
```

```
    // Apply a 2D transformation
```

```
    ctx.transform(1, 0.5, 0.5, 1, 30, 10);
```

```
    // Draw a square after transformation
```

```
    ctx.fillStyle = 'red';
```

```
    ctx.fillRect(50, 70, 50, 50);
```

```
}
```

```
</script>
```

```
</head>
```

```
<body onload="displayQuotes();">
```

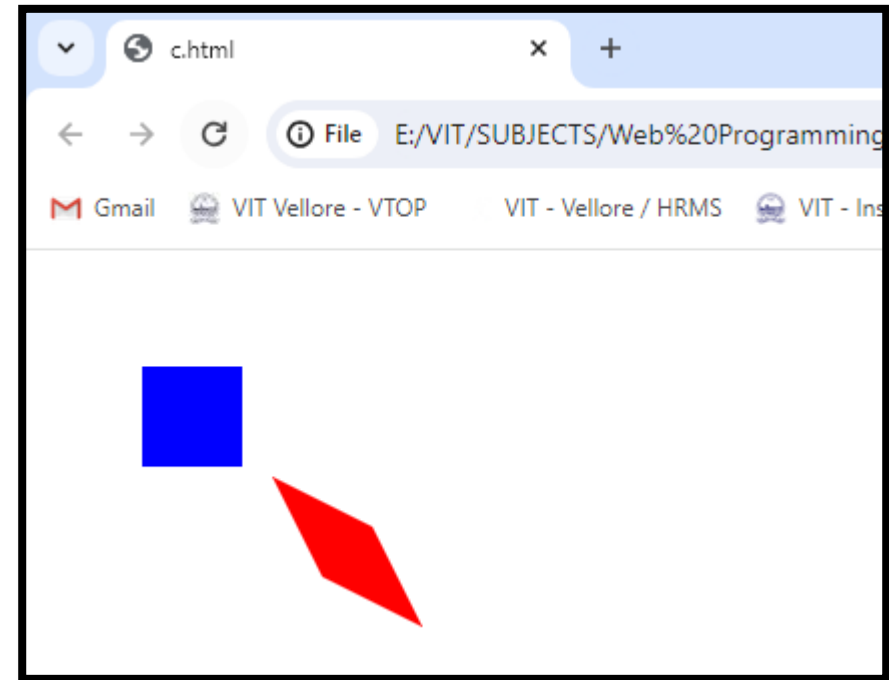
```
<canvas id="canvas" width="600" height="400">
```

```
</canvas>
```

```
</body>
```

```
</html>
```

It modifies the existing transformation matrix by multiplying it with the new matrix specified by the parameters.



setTransform()

- This method is used to set a new transformation matrix that includes horizontal scaling, horizontal skewing, vertical skewing, vertical scaling, horizontal translation, and vertical translation.

`ctx.setTransform(2, 0.5, -0.5, 1.5, 30, 10)`

- **a (m11):** Horizontal scaling by a factor of **2**.
- **b (m21):** Horizontal skewing by **0.5**.
- **c (m12):** Vertical skewing by **-0.5**.
- **d (m22):** Vertical scaling by a factor of **1.5**.
- **e (dx):** Horizontal translation by **30** units.
- **f (dy):** Vertical translation by **10** units.

Example:

```
<!DOCTYPE html><head>
```

```
<script>
```

```
function displayQuotes() {
```

```
    var ctx; // the canvas object's context
```

```
    ctx = document.getElementById("canvas").getContext("2d");
```

```
    // Draw a square before transformations
```

```
    ctx.fillStyle = 'blue';
```

```
    ctx.fillRect(50, 50, 50, 50);
```

```
    // Apply a set of transformations using setTransform
```

```
    ctx.setTransform(2, 0.5, -0.5, 1.5, 30, 10);
```

```
    // Draw a square after transformations
```

```
    ctx.fillStyle = 'red';
```

```
    ctx.fillRect(50, 50, 50, 50);
```

```
}
```

```
</script>
```

```
</head>
```

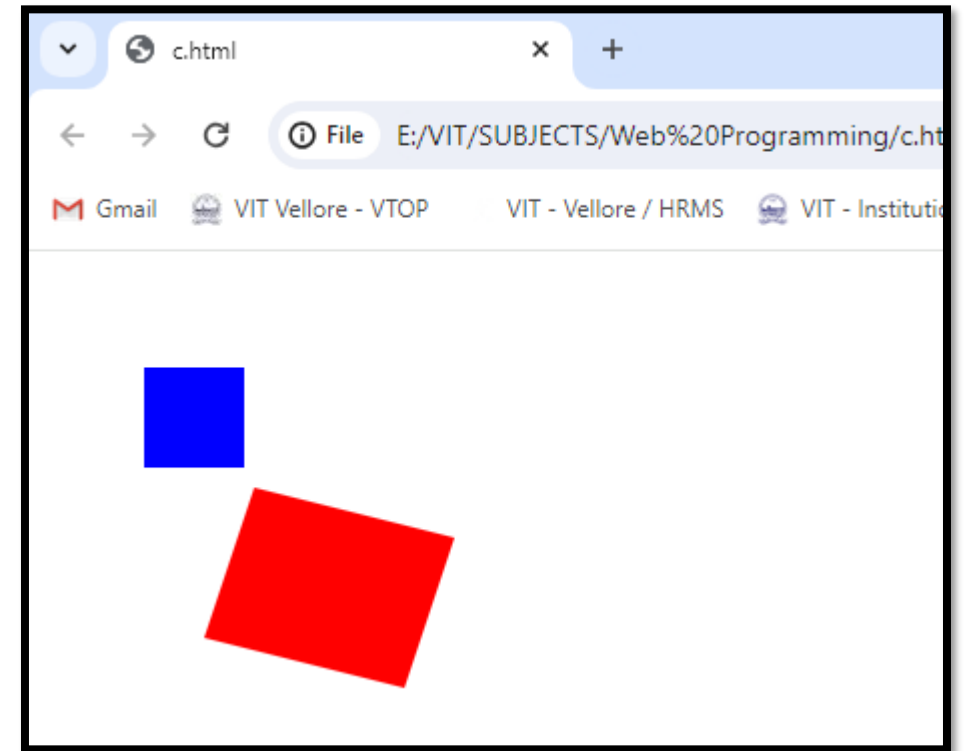
```
<body onload="displayQuotes();">
```

```
<canvas id="canvas" width="600" height="400">
```

```
</canvas>
```

```
</body>
```

```
</html>
```



JavaScript Strings

JavaScript Strings

- JavaScript strings are for **storing and manipulating text**.
- A JavaScript string is zero or more characters written inside quotes.

let text = "Hiii Studdents";

- can use **single or double quotes**

```
<!DOCTYPE html>
<html>
<body>

<h1>Strings</h1>

<p id="demo"></p>

<script>
let text = "Hello students!!!!"; // String written inside quotes
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

JavaScript Strings

Hello students!!!!

JavaScript Strings

String Length

- To find the length of a string, use the built-in length property

```
<!DOCTYPE html>
<html>
<body>

<h2>The length Property</h2>

<p>The length of the string is:</p>
<p id="demo"></p>

<script>
let text = "The best way to predict your future is to create it.";
document.getElementById("demo").innerHTML = text.length;
</script>

</body>
</html>
```

The length Property

The length of the string is:

52

String Methods

Extracting String Parts

- There are 3 methods for extracting a part of a string:
 - `slice(start, end)`
 - `substring(start, end)`
 - `substr(start, length)`

String slice()

- slice() extracts a part of a string and returns the extracted part in a new string.
- The method takes 2 parameters:
 - the start position, and
 - the end position (end not included).
- If a parameter is negative, the position is counted from the end of the string.

String slice()

```
<!DOCTYPE html>
<html>
<body>

<h2>The slice() Method</h2>

<p>The sliced part of the string is:</p>
<p id="demo"></p>

<script>
let text = "The best way to predict your future is to create it.";
let part = text.slice(16,23);
document.getElementById("demo").innerHTML = part;
</script>

</body>
</html>
```

The slice() Method

The sliced part of the string is:

predict

substring()

- `substring()` is similar to `slice()`.
- The `substring()` method of String values returns the part of this string from the start index up to and excluding the end index, or to the end of the string if no end index is supplied.

Syntax:

`substring(indexStart)`

`substring(indexStart, indexEnd)`

substring()

```
<!DOCTYPE html>
<html>
<body>

<p>The substring() method extract a part of a string and returns the
extracted parts in a new string:</p>

<p id="demo"></p>

<script>
let str = "The best way to predict your future is to create it.";
document.getElementById("demo").innerHTML = str.substring(4,13);
</script>

</body>
</html>
```

JavaScript String Methods

The substring() method extract a part of a string and returns the extracted parts in a new string:

best way

substr()

- substr() is similar to slice().
- The difference is that the second parameter specifies the length of the extracted part.
- If you omit the second parameter, substr() will slice out the rest of the string.

substr()

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript String Methods</h1>

<p id="demo"></p>

<script>
let str = "The best way to predict your future is to create it.";
document.getElementById("demo").innerHTML = str.substr(2,9);
</script>

</body>
</html>
```

JavaScript String Methods

e best wa

substr()

- If you omit the second parameter, substr() will slice out the rest of the string.

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript String Methods</h1>

<p id="demo"></p>

<script>
let str = "The best way to predict your future is to create it.";
document.getElementById("demo").innerHTML = str.substr(2);
</script>

</body>
</html>
```

JavaScript String Methods

e best way to predict your future is to create it.

Replacing String Content

- The replace() method **replaces a specified value with another value in a string**
- The replace() method **does not change the string it is called on.**
- The replace() method **returns a new string.**
- The replace() method **replaces only the first match**
- By default, the replace() method is **case sensitive.**

Replacing String Content

```
<!DOCTYPE html>
<html>
<body>

<p id="demo">Fear leads to anger; anger leads to hatred; hatred leads to conflict;
conflict leads to suffering.</p>

<script>
let str = document.getElementById("demo").innerHTML;
let res = str.replace("anger", "JAVASCRIPT");
document.getElementById("demo").innerHTML = res;
</script>

</body>
</html>
```

Fear leads to JAVASCRIPT; anger leads to hatred; hatred leads to conflict; conflict leads to suffering."

Converting to Upper and Lower Case

HELLO WORLD!

- `let text1 = "Hello World!";`
- `let text2 = text1.toUpperCase();`

hello world!

- `let text1 = "Hello World!";`
- `let text2 = text1.toLowerCase();`

String concat()

- concat() joins two or more strings

```
let text1 = "Hello";
```

```
let text2 = "World";
```

```
let text3 = text1.concat(" ", text2);
```

Hello World!

- The concat() method can be used instead of the plus operator. These two lines do the same:

```
text = "Hello" + " " + "World!";
```

```
text = "Hello".concat(" ", "World!");
```

- All string methods return a new string. They don't modify the original string.

String trim()

- The trim() method **removes whitespace from both sides of a string**

```
<!DOCTYPE html>
<html>
<body>

<pre><p id="demo1"></p></pre>
<pre><p id="demo2"></p></pre>

<script>
let text = "      Fear leads to anger      ";
let result = text.trim();

document.getElementById("demo1").innerHTML = text;
document.getElementById("demo2").innerHTML = result;
</script>

</body>
</html>
```

Fear leads to anger

Fear leads to anger

Extracting String Characters

The `charAt()` method **returns the character at a specified index (position) in a string**

```
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>

<script>
let text = "Fear leads to anger; anger leads to hatred; hatred leads to
conflict; conflict leads to suffering.";

let string1 = text.charAt(2);

document.getElementById("demo").innerHTML = string1;
</script>

</body>
</html>
```

a

Extracting String Characters

charCodeAt()

- The charCodeAt() method **returns the unicode of the character at a specified index in a string**
- The method returns a **UTF-16 code (an integer between 0 and 65535)**.

```
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>

<script>
let text = "Fear leads to anger; anger leads to hatred; hatred leads to
conflict; conflict leads to suffering.";

let string1 = text.charCodeAt(2);

document.getElementById("demo").innerHTML = string1;
</script>

</body>
</html>
```

String split()

- A string can be converted to an array with the split() method

```
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>
<p id="demo1"></p>
<p id="demo2"></p>

<script>

let text = "Fear leads to anger; anger leads to hatred; hatred leads to conflict;
conflict leads to suffering.";

let string1 = text.split(" ");
document.getElementById("demo").innerHTML = string1;

let string2 = text.split(" ",4);
document.getElementById("demo1").innerHTML = "Using limit parameter: "+string2;

let string3 = text.split(";");
document.getElementById("demo2").innerHTML = string3;

</script>

</body>
</html>
```

Fear,leads,to,anger;,anger,leads,to,hatred;,hatred,leads,to,conflict;,conflict,leads,to,suffering.

Using limit parameter: Fear,leads,to,anger;

Fear leads to anger, anger leads to hatred, hatred leads to conflict, conflict leads to suffering.